

Methodology

Project Title

SmartNShine ATS Resume Generator with AI Interview Practice

Date

13 March 2026

1. Introduction

This document describes the development methodology adopted for the SmartNShine platform. It covers the software development lifecycle approach, tooling choices, design decisions, implementation strategy, testing philosophy, security practices, and deployment approach. The methodology was shaped by the project's scope — a full-stack MERN application with multiple integrated AI services, voice microservices, payment processing, and a role-based access system.

The goal of this document is to provide a clear account of how the project was planned, built, and validated, and to justify the technical and process decisions made throughout development.

2. Development Approach

2.1 Methodology Model — Iterative and Incremental Development

SmartNShine was built using an **iterative and incremental development** approach, drawing from core Agile principles. Rather than designing the full system up front before writing any code, the platform was developed in focused feature cycles, each of which produced a working and testable vertical slice of the system.

This approach was chosen for the following reasons:

1. The platform integrates multiple external APIs (Gemini, OpenAI, Razorpay, OAuth, Whisper, Chatterbox) whose real behaviors are best understood through implementation rather than speculation.
2. UI/UX quality for AI-driven features — such as the resume editor and interview interface — requires tight feedback cycles between build and review.
3. Some features such as voice interview and advanced ATS scoring required direct experimentation to establish working configurations.
4. Scope changes, such as replacing ElevenLabs with Chatterbox, occurred mid-development and needed to be absorbed without disrupting the overall system.

2.2 Development Cycles

Development proceeded through the following broad phases, each iterated upon before the next phase was heavily committed to:

Phase	Focus
Phase 1	Project scaffolding, authentication, user model, JWT, OAuth
Phase 2	Resume upload, parsing, structured editor, templates
Phase 3	AI integration — Gemini for parsing/enhancement, ATS scoring
Phase 4	Subscription system, Razorpay payment flow, plan-gated features
Phase 5	AI interview module — session management, question generation, evaluation
Phase 6	Voice pipeline — Whisper STT, Chatterbox TTS, browser fallback
Phase 7	Admin panel — user management, analytics, logs, templates, settings
Phase 8	Hardening — rate limiting, security middleware, error handling, audit

Each phase was treated as a mini-project: design the feature, implement the backend service and API, build the frontend page or component, test manually, and then move on. After completing all phases, a cross-cutting review was performed to address security, consistency, and code quality.

3. Architecture and Design Methodology

3.1 Architecture Decision — Monorepo with Microservices for Voice

The project follows a **monorepo layout** with `client/`, `server/`, `voice-service/`, and `chatterbox-service/` directories managed from a single root `package.json`. This provides a single repository for the full system while still isolating each service.

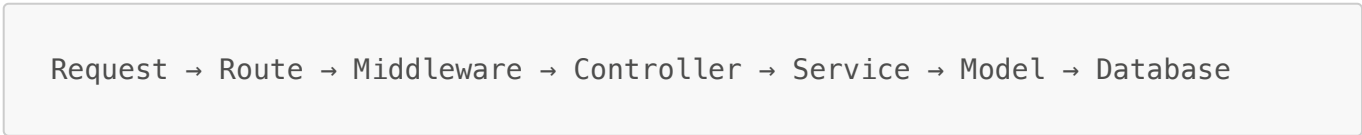
Voice services are separated into independent Python Flask microservices for the following reasons:

- OpenAI Whisper and Chatterbox TTS require Python runtimes and GPU-compatible environments that are incompatible with the Node.js backend runtime.
- Separating these services allows them to be independently started, restarted, and scaled.
- Health check endpoints on each service allow the main backend to degrade gracefully when a voice service is unavailable.

The Node.js Express server communicates with voice services via HTTP proxy routes, so the React frontend never calls the Python services directly. This keeps authentication and CORS policies centralized on the Express layer.

3.2 Backend Design — MVC with Service Layer

The Express backend follows a **Model-View-Controller (MVC) pattern extended with a service layer**:



- **Routes** map HTTP methods and paths to controller functions.

- **Middleware** handles authentication, rate limiting, file upload, and request validation before the controller is reached.
- **Controllers** receive the validated request, call the relevant service, and return the HTTP response.
- **Services** contain all business logic, AI calls, and database interactions. They are independent of Express request/response objects, making them testable in isolation.
- **Models** define Mongoose schemas and handle data validation at the persistence layer.

This separation ensures that no business logic leaks into route handlers, and that controllers remain thin and readable.

3.3 Frontend Design — Component-Driven with Hooks

The React frontend was built following a **component-driven architecture**:

- UI is broken into small, reusable components organized under **components/** by domain: **admin/**, **auth/**, **common/**, **editor/**, **layout/**, **templates/**.
- Page-level components under **pages/** compose these smaller components and manage page-specific state.
- Custom hooks under **hooks/** extract reusable stateful logic such as debouncing, navigation blocking, and ATS score calculation away from component bodies.
- Context providers (**DarkModeContext**, **NavigationBlockerContext**) manage global state without requiring a full Redux setup.
- API calls are centralized under **api/** as Axios client modules, keeping service communication decoupled from component logic.

TailwindCSS was used for all styling. No CSS modules or styled-components were introduced to keep the styling approach uniform and the build pipeline simple.

3.4 Database Design Methodology

MongoDB was selected as the database for the following reasons:

1. Resume content is highly variable in structure — sections, items, and ordering differ per user. A document model accommodates this naturally without schema migration complexity.
2. AI service responses are JSON objects that embed cleanly into MongoDB documents.
3. The MERN stack standardizes JavaScript/JSON from frontend to database, reducing data transformation overhead.

Schema design followed these principles:

- Each major entity (User, Resume, InterviewSession, InterviewResult, Subscription, etc.) has its own Mongoose model file with inline validation.
- Related entities are **referenced by ID** (not embedded) when the relationship is one-to-many or when the sub-document could grow unboundedly (e.g., interview sessions per user).
- Frequently co-accessed data such as resume sections are **embedded** within the Resume document to reduce query round-trips.
- Indexes are applied on commonly queried fields such as **user** foreign keys and **status** fields.

4. AI Integration Methodology

4.1 AI Model Selection

Two AI providers are used, each chosen for specific task characteristics:

Provider	Model	Use Cases
Google Gemini	<code>gemini-1.5-flash</code> / <code>gemini-pro</code>	Resume parsing, enhancement, summarization, ATS scoring, skill categorization
OpenAI	<code>gpt-4o</code>	Interview question generation, answer evaluation, result report generation

Gemini was selected for resume-related tasks because it performs well on structured extraction tasks (parsing resume text into JSON), handles longer context windows efficiently, and is cost-effective for high-frequency operations. OpenAI GPT-4o was selected for interview evaluation because it produces consistently structured and nuanced feedback suited to multi-dimensional scoring prompts.

4.2 Prompt Engineering Strategy

All AI prompts are maintained in the service layer (`server/services/`) and follow consistent structure principles:

- Role assignment** — the model is given an explicit role (e.g., "You are an expert resume parser") to establish the response tone.
- Structured output requirement** — prompts explicitly request JSON output with defined schemas, reducing post-processing complexity.
- Input context injection** — the relevant data (resume text, job description, previous question) is embedded in the prompt template at call time.
- Temperature tuning** — question generation uses a higher temperature (0.7) for diversity; evaluation uses a lower temperature for consistency.
- Error resilience** — service functions parse AI responses with fallback handling for malformed JSON or unexpected response shapes.

4.3 AI Usage Quota Management

AI operations are gated through a quota system stored in the `AIUsage` model:

- Each AI operation type (enhancement, ATS scan, interview question, etc.) has a per-user daily or monthly limit configured by subscription tier.
- Before executing an AI call, the relevant controller checks the current usage against the quota.
- On exceeding quota, a `429` response is returned with an informative message rather than silently degrading.
- Admins can view and reset AI usage statistics from the admin dashboard.

5. Voice Pipeline Methodology

5.1 Service Architecture Decision

Voice services were implemented as separate Python Flask microservices because:

- OpenAI Whisper requires Python and optionally CUDA for GPU acceleration.
- Chatterbox TTS is a Python-native library with no Node.js binding.
- Running Python processes inside a Node.js server would require brittle subprocess management.

Each microservice exposes a minimal REST API with a `/health` endpoint and function-specific endpoints. The Node.js backend proxies voice requests from the frontend to these services, adding authentication and error handling in the proxy layer.

5.2 Fallback Chain Design

Voice output reliability is addressed through a **priority fallback chain**:

```
Chatterbox TTS (port 5002)
  ↓ (on failure or unavailability)
Browser Web Speech API (client-side, no server required)
```

This ensures the interview feature remains functional even when the local Python service is not running, which is especially important in development and demo environments.

Speech-to-text similarly checks Whisper service availability before accepting voice input, with the service health result surfaced in the UI so users understand the active mode.

5.3 Voice Service Integration Testing

Because voice services are external processes, their integration is validated through health check calls at application startup and on-demand before voice-dependent operations. This avoids runtime errors mid-interview and provides early warning of configuration problems.

6. Security Methodology

6.1 Authentication

- JWT-based authentication is applied to all protected API routes via an `authMiddleware`.
- Tokens are signed with a server-side secret stored in environment variables.
- OAuth login (Google and GitHub) is handled through Passport.js strategies, which issue the same JWT token after OAuth callback.
- Passwords are hashed using bcrypt before storage. Plain-text passwords are never persisted.

6.2 Authorization

- Role-based access control is enforced at the route and middleware level. Admin-only routes verify `req.user.role === 'admin'` before allowing access.
- Subscription-gated features verify active subscription status before executing premium operations.
- Users can only access their own resume, session, and subscription data. Queries always include the authenticated user's ID as a filter condition.

6.3 Input Security

- Mongoose schema validation provides first-level input sanitization at the model layer.
- `express-mongo-sanitize` middleware prevents NoSQL injection attacks by stripping `$` and `.` operators from request bodies.
- `helmet` middleware sets security-related HTTP headers on all responses.
- File upload validation (via Multer configuration) restricts accepted MIME types and file sizes.
- Rate limiting is applied to authentication, AI, file upload, and voice endpoints using `express-rate-limit`.

6.4 Secrets Management

- All API keys, OAuth credentials, JWT secrets, and database connection strings are stored in `.env` files that are excluded from version control via `.gitignore`.
- No credentials are hardcoded in source files.

7. Testing Methodology

7.1 Testing Approach

Given the project's scope and the AI-heavy nature of its core features, the testing strategy is primarily **manual integration testing** supplemented by **API-level verification**. Automated unit testing was not implemented at this stage, which is noted as a future improvement area.

Testing was performed in the following ways:

1. **Feature-level manual testing** — each feature was tested end-to-end by exercising the UI as a user would, verifying the expected behavior and checking MongoDB state for consistency.
2. **API testing using REST client tools** — individual API routes were tested in isolation using tools like Postman or equivalent, verifying request/response contracts, authentication enforcement, error codes, and edge cases.
3. **AI response validation** — AI service responses were reviewed manually for structural correctness, with prompt iterations performed where outputs were inconsistent or malformed.
4. **Voice pipeline testing** — voice features were tested by verifying health endpoints, recording test audio, and observing the transcription and TTS playback pipeline.
5. **Cross-role testing** — the application was tested under guest, registered user, subscriber, and admin roles to verify access control enforcement.
6. **Edge case testing** — boundary conditions such as expired subscriptions, quota exhaustion, unavailable AI services, and malformed file uploads were tested to ensure graceful degradation.

7.2 Defect Tracking

Issues identified during testing were documented in dedicated audit and fix log files within the repository. Examples include `VOICE_SERVICE_AUDIT.md` for voice-related bugs, `ATS_RESUME_FEATURE_BUGS_AND_ENHANCEMENTS.md` for editor issues, and other feature-specific documentation. This provides traceability from identified bug to applied fix.

8. Version Control Methodology

8.1 Git Branching Strategy

The project uses a **feature branch workflow** based on Git:

- **main** — stable, reviewed code representing the latest working state.
- **ai-interview** — the active development branch for AI interview features, branched from **main** and periodically merged back.
- Feature-level work is committed incrementally with descriptive commit messages following a conventional format (e.g., **feat:**, **fix:**, **refactor:**).

8.2 Commit Discipline

Commits were made at logical checkpoints — completing a feature, fixing a bug, or refactoring a module — rather than at arbitrary save points. This keeps the git history readable and makes it straightforward to identify when a behavior was introduced or changed.

9. Tooling

9.1 Development Tools

Tool	Purpose
VS Code	Primary code editor
Git + GitHub	Version control and remote repository
Postman	API endpoint testing
MongoDB Compass	Database inspection and query testing
Chrome DevTools	Frontend debugging and network inspection
Vite	Frontend development server and build tool

9.2 Key Libraries and Frameworks

Category	Library / Framework
Frontend framework	React 18
Build tool	Vite 6
Styling	TailwindCSS 3
Animation	Framer Motion
Rich text editing	TipTap
HTTP client	Axios
Backend framework	Express 4
Database ORM	Mongoose 8

Category	Library / Framework
Authentication	Passport.js + JWT
AI — resume	@google/generative-ai (Gemini)
AI — interview	openai (GPT-4o)
Payment	Razorpay Node SDK
File upload	Multer
Security	Helmet, express-mongo-sanitize, express-rate-limit
Email	Nodemailer
Voice STT	Python Flask + OpenAI Whisper
Voice TTS	Python Flask + Chatterbox
Concurrency	concurrently (dev script runner)

9.3 Environment Configuration

Environment variables are used to configure all external service credentials and runtime settings, with separate `.env` files for the client and server. This allows the application to be deployed to different environments (development, staging, production) by changing environment configuration without modifying code.

10. Deployment Considerations

The current codebase is structured to support the following deployment topology:

- **Frontend** — buildable as a static asset bundle via `vite build`, deployable to any static hosting provider or CDN.
- **Backend** — a standard Node.js process deployable to any Node.js-compatible server or platform service.
- **Voice services** — Python Flask processes with their own `requirements.txt` and startup scripts, containerizable via the provided `Dockerfile` for the Chatterbox service.
- **Database** — MongoDB Atlas cloud cluster, accessed via connection string environment variable.

For local development, `concurrently` is used to start the frontend and backend simultaneously from the root `package.json`. Voice services are started separately using their respective `start.sh` scripts.

A production deployment would require:

1. Setting all environment variables for the target environment.
2. Building the frontend and serving it via a CDN or Nginx static host.
3. Running the Express server behind a reverse proxy (e.g., Nginx) with HTTPS termination.
4. Running voice microservices on a Python-capable host, optionally using Docker for isolation.

11. Challenges and How They Were Addressed

Challenge	Resolution
ElevenLabs TTS became unavailable due to payment issues	Integrated Chatterbox TTS as server-side primary, with browser Web Speech API as client-side fallback
AI responses sometimes returned malformed JSON	Added structured output instructions to prompts and defensive JSON parsing with fallback shapes
Voice interview required coordinating STT, TTS, and UI state simultaneously	Implemented a priority-based health check system and separated voice state management into dedicated state variables
Subscription access control needed to interact with AI quota checks consistently	Centralized quota enforcement logic into service-layer utility functions called before every AI operation
Resume sections vary widely in structure per user	Used flexible embedded document arrays with optional fields rather than rigid normalized tables
Large interview page (~3300 lines) became difficult to maintain	Applied component extraction and custom hook patterns iteratively to isolate manageable concerns

12. Limitations of the Current Methodology

1. The absence of automated unit and integration tests means regression detection relies on manual re-testing after changes.
2. Some features are only partially tested due to the cost and availability constraints of external APIs in development.
3. The `ai-interview` branch includes work-in-progress features that have not been fully merged to `main`, meaning the main branch and active branch diverge on certain capabilities.
4. Voice service reliability testing is environment-dependent since Whisper and Chatterbox require specific hardware and Python environments not universally available.

13. Future Methodology Improvements

1. Introduce Jest-based unit testing for service layer functions, particularly AI prompt builders and response parsers.
2. Add Supertest-based integration tests for key API route flows.
3. Set up a CI/CD pipeline (e.g., GitHub Actions) to run lint and test suites on every pull request.
4. Containerize the full stack using Docker Compose to standardize the development environment and eliminate voice service setup friction.
5. Introduce end-to-end testing using Playwright or Cypress for critical user journeys such as registration, resume editing, ATS scan, and interview completion.

14. Conclusion

The methodology for SmartNShine was designed to balance practicality with quality for a complex multi-service platform. Iterative development enabled continuous integration of AI and voice services whose

behaviors were refined through experimentation. The MVC with service layer backend, component-driven frontend, and microservice-based voice pipeline each provided clear separation of concerns that allowed features to be added and modified independently. Security, quota management, and graceful degradation were treated as first-class concerns rather than afterthoughts. While the testing approach is currently manual, the architecture is structured to support the addition of automated testing and CI/CD pipelines as the platform matures.